

- คำตอบที่ใช่ได้: "เลือกให้เหมาะกับขนาดงานและงบประมาณของเทศบาล ถ้าต้องขยายเป็นระบบใหญ่ในอนาคต ค่อยย้ายไปใช้ฐานข้อมูลจริงได้ เพราะโครงสร้าง (แยกหน้าเว็บ/ตัวกลาง/ที่เก็บข้อมูล) ออกแบบไว้รองรับการย้ายอยู่แล้ว"

3. เทคโนโลยีที่ใช้ และทำหน้าที่อะไร

เทคโนโลยี	ทำหน้าที่อะไร	ทำไมเลือกตัวนี้
HTML	โครงหน้าเว็บ (ปุ่ม, ฟорм, ตาราง)	มาตรฐานพื้นฐานของเว็บทุกหน้า
CSS (Tailwind ผ่าน CDN)	จัดสี/จัดวาง ให้สวยและใช้บนมือถือได้	เขียนสไตส์เร็ว ไม่ต้องติดตั้งอะไรเพิ่ม แค่แปะ <code><script></code> บรรทัดเดียว
JavaScript (Vanilla)	สั่งงานตอนกดปุ่ม, ค้นหา, validation, เรียกข้อมูล	ไม่ใช่ framework (เช่น React) เพื่อให้โค้ดอ่านง่าย ไม่ต้อง build
Google Sheets	เก็บข้อมูลผู้ป่วย/ผู้ใช้/ค่าตั้งค่า แทนฐานข้อมูล	ฟรี, มี UI ให้ดูข้อมูลตรงๆ, ทุกคนคุ้นเคยอยู่แล้ว
Google Apps Script	รับคำขอจากหน้าเว็บ (เช่น "ขอรายชื่อผู้ป่วย") แล้วไปอ่าน/เขียน Google Sheet ให้	เป็นวิธีมาตรฐานที่ Google ให้มาเชื่อม Sheet กับเว็บภายนอก ทำงานเหมือน "backend API" เล็กๆ
Google Maps URL Scheme	สร้างลิงก์เปิดแอป Google Maps พร้อมจุดเริ่ม-จุดหมาย	ใช้ฟรี ไม่ต้องขอ API Key หรือผูกบัตรเครดิตเหมือน Maps JavaScript API

4. ระบบทำงานอย่างไร (ยกตัวอย่าง: ตอนกดปุ่มนำทาง)

1. ผู้ใช้เปิดหน้า `patient-detail.html?id=P-2026-0001` → JavaScript ยิง request 2 คำขอพร้อมกัน คือขอข้อมูลผู้ป่วยคนนั้น (`action=get`) และขอค่าตั้งค่าเพื่อเอาพิกัดเทศบาล (`action=getSettings`) ยิงพร้อมกันด้วย `Promise.all` เพื่อไม่ต้องรอทีละคำขอ หน้าเว็บจะขึ้นเร็วกว่า
2. Apps Script รับคำขอ → เปิด Google Sheet → หาแถวที่ตรงกับ `patient_code` → ส่งข้อมูลกลับเป็น JSON
3. JavaScript เอาข้อมูลมาแสดงผลบนหน้าเว็บ (ชื่อ, ที่อยู่, พิกัด)
4. ตอนที่หน้าเว็บวาดเสร็จ JavaScript สร้างลิงก์ Google Maps ใส่ไว้ในปุ่มนำทางรอไว้เลย โดยใช้พิกัดเทศบาลเป็นจุดเริ่มและพิกัดผู้ป่วยเป็นจุดหมาย → ผู้ใช้กดปุ่มก็เปิดแอปได้ทันที (ถ้าข้อมูลไม่พอสร้างลิงก์ ปุ่มจะถูกปิดการใช้งาน พร้อมข้อความบอกสาเหตุ)

ตัวอย่างการยิง request จริง (อธิบายง่ายๆ ได้ว่า "หน้าเว็บส่งข้อความไปถาม แล้วรอคำตอบกลับมา"):

```
หน้าเว็บ → GET https://script.google.com/macros/s/xxxxx/exec?action=list&token=xxxx
Apps Script → ตอบกลับเป็น JSON: { ok: true, data: [{ patient_code: "P-2026-0001", first_name: "สมชาย", ... }] }
```

ลิงก์นำทางที่ระบบสร้างขึ้นจริง (คัดลอกจากการทดสอบ ใช้โชว์ได้เลย):

```
https://www.google.com/maps/dir/?api=1
&origin=13.736717,100.523186 ← พิกัดเทศบาล อ่านจากแท็บ Settings
&destination=13.740000,100.530000 ← พิกัดบ้านผู้ป่วย
&travelmode=driving
```

ถ้าผู้ป่วยไม่มีพิกัด ระบบจะใส่ที่อยู่เป็นข้อความลงใน `destination` แทน (ผ่าน `encodeURIComponent`)

4.1 คำสั่ง (action) ทั้งหมดที่ระบบมี

หน้าเว็บคุยกับ Apps Script ผ่าน "คำสั่ง" 9 คำสั่งเท่านั้น ไม่มีทางอื่น

คำสั่ง	วิธีส่ง	ต้องมีสิทธิ์	ทำอะไร
<code>login</code>	POST	—	ตรวจ username/password กับแท็บ Users แล้วคืน token
<code>list</code>	GET	ล็อกอินแล้ว	ดึงรายชื่อผู้ป่วยทั้งหมด (หรือเฉพาะที่ถูกลบ ถ้าส่ง <code>includeDeleted=1</code>)
<code>get</code>	GET	ล็อกอินแล้ว	ดึงข้อมูลผู้ป่วย 1 คนจาก <code>patient_code</code>
<code>create</code>	POST	admin	เพิ่มผู้ป่วยใหม่ + ออกรหัสให้อัตโนมัติ
<code>update</code>	POST	admin	แก้ไขข้อมูลผู้ป่วย
<code>delete</code>	POST	admin	Soft Delete — ใส่เวลาลงคอลัมน์ <code>deleted_at</code>
<code>restore</code>	POST	admin	กู้คืน — ล้างค่า <code>deleted_at</code>
<code>getSettings</code>	GET	—	อ่านค่าตั้งค่าจากแท็บ Settings
<code>updateSettings</code>	POST	admin	บันทึกชื่อระบบ / พิกัดเทศบาล

ทำไม `getSettings` ไม่ต้องล็อกอิน? เพราะหน้า `login.html` ต้องใช้ชื่อระบบมาแสดงตั้งแต่ก่อนที่ผู้ใช้จะล็อกอิน และข้อมูลในแท็บ Settings ไม่ใช่ข้อมูลส่วนบุคคลของผู้ป่วย

4.2 ระบบล็อกอินทำงานยังไง

- ผู้ใช้กรอก username/password → หน้าเว็บส่งไปที่ Apps Script (`action=login`)
- Apps Script อ่านแท็บ `Users` หาแถวที่ username ตรงกัน แล้วเทียบรหัสผ่าน และตรวจว่าคอลัมน์ `is_active` เป็น `TRUE`
- ถ้าผ่าน Apps Script สร้าง **token** (รหัสสุ่มยาวๆ ที่เดาไม่ได้ ด้วย `Utilities.getUuid()`) แล้วเก็บไว้ใน `CacheService` ของ Google พร้อมกำหนดอายุ **6 ชั่วโมง**
- หน้าเว็บเก็บ token นั้นไว้ใน `sessionStorage` ของเบราว์เซอร์ (หายเมื่อปิดแท็บ)

5. ทุกคำสั่งหลังจากนั้นต้องแนบ token ไปด้วย — Apps Script เอา token ไปเทียบใน Cache ถ้าไม่เจอ (หมดอายุ) จะตอบกลับว่า "เซสชันหมดอายุแล้ว กรุณาเข้าสู่ระบบใหม่"

จุดที่ควรพูดถึงถ้าอาจารย์ถามเรื่องความปลอดภัย: การตรวจสอบสิทธิ์ทำ 2 ชั้น

- ชั้นหน้าเว็บ (`Auth.requireLogin('admin')`) — ซ่อนเมนูและดึงออกจากหน้าที่ไม่มีสิทธิ์
- ชั้นเซิร์ฟเวอร์ (`requireAuth_(params, 'admin')` ใน Code.gs) — ต่อให้ผู้ใช้แก้โค้ดฝั่งหน้าเว็บ หรือยิงคำสั่งเข้ามาตรงๆ ก็ยังเขียนข้อมูลไม่ได้ ถ้า token ไม่ใช่ของ admin

การซ่อนปุ่มฝั่งหน้าเว็บอย่างเดียวไม่ถือว่าปลอดภัย — นี่คือเหตุผลที่ต้องเช็คซ้ำที่ฝั่งเซิร์ฟเวอร์ด้วย

5. โครงสร้างไฟล์ในโปรเจกต์

```
patient-records-simple/
├── SPEC.md           ข้อกำหนดระบบ (ขอบเขตงานทั้งหมด)
├── CLAUDE.md         กติกาการเขียนโค้ดของโปรเจกต์
├── PROJECT_STATUS.md สถานะงาน + บันทึกการตัดสินใจ
├── design-document.md ไฟล์นี้ - อธิบายการออกแบบระบบ
├── docs/
│   └── user-manual.md คู่มือการใช้งานฉบับเอกสาร
├── images/
│   └── logo.png      ตราสัญลักษณ์เทศบาล (ไฟล์ static)
├── style.css         สไตล์เสริมจาก Tailwind (ตัวแปรสี ปุ่ม การ์ด ช่องกรอก)
├── login.html        หน้าเข้าสู่ระบบ
├── index.html        หน้ารายชื่อผู้ป่วย + ค้นหา
├── patient-detail.html หน้ารายละเอียด + ปุ่มนำทาง
├── patient-form.html หน้าเพิ่ม/แก้ไขผู้ป่วย
├── trash.html        ถังขยะ / กู้คืนข้อมูล
├── settings.html     ตั้งค่าระบบ (เฉพาะ admin)
├── help.html         คู่มือการใช้งานในเว็บ
├── js/
│   ├── api.js        ตัวกลางเรียก Apps Script (จุดเดียวที่มี URL ปลายทาง) + ข้อมูลจำลองโหมดเดโม
│   ├── auth.js       เช็คว่าล็อกอินหรือยัง + ควบคุมสิทธิ์ + สร้างแถบเมนูที่ใช้ร่วมกันทุกหน้า
│   ├── patients.js   โหลด/ค้นหา/กรองรายชื่อผู้ป่วย + ฟังก์ชันสร้างลิงก์นำทาง
│   └── patient-form.js validation + ส่งข้อมูลฟอร์ม + ดึงพิกัดจากลิงก์ Google Maps
└── apps-script/
    └── Code.gs        โค้ดฝั่ง Google Apps Script (backend)
```

จุดที่น่าสนใจในการออกแบบไฟล์:

- แถบเมนูด้านบนเขียนไว้ที่เดียวใน `js/auth.js` (ฟังก์ชัน `renderHeader`) ทุกหน้าเรียกใช้ร่วมกัน แก้มenuหรือโลโก้ที่เดียว เปลี่ยนพร้อมกันทั้ง 6 หน้า — เป็นหลักการ DRY (Don't Repeat Yourself)
- `js/api.js` เป็นไฟล์เดียวที่รู้จัก URL ของ Apps Script ไฟล์อื่นเรียกผ่านฟังก์ชัน `Api.xxx()` ถ้าวันหลังเปลี่ยนไปใช้ฐานข้อมูลอื่น แก่แค่ไฟล์นี้ไฟล์เดียว

- `apps-script/Code.gs` ไม่ได้ทำงานจากโฟลเดอร์นี้ เพราะ Apps Script deploy ผ่านเว็บ ไฟล์ในโปรเจกต์เก็บไว้เป็นต้นฉบับสำหรับคัดลอกไปวาง

5.1 โครงสร้างข้อมูลใน Google Sheet

Google Sheet 1 ไฟล์ มี 3 แท็บ เทียบได้กับ "3 ตาราง" ในฐานข้อมูลทั่วไป

แท็บ `Patients` — 14 คอลัมน์

คอลัมน์	ชนิดข้อมูล	หมายเหตุ
<code>patient_code</code>	ข้อความ	รหัสประจำตัว เช่น <code>P-2026-0001</code> ระบบออกให้เอง ใช้เป็นตัวระบุตัวตน (Primary Key)
<code>first_name</code> / <code>last_name</code>	ข้อความ	บังคับกรอก
<code>patient_type</code>	ข้อความ	<code>bedridden</code> (ติดเตียง) หรือ <code>general</code> (ทั่วไป)
<code>address_text</code>	ข้อความ	บังคับกรอก ใช้นำทางสำรองถ้าไม่มีพิกัด
<code>symptoms</code>	ข้อความ	อาการ/โรคประจำตัว บังคับกรอก
<code>phone</code> / <code>note</code>	ข้อความ	ไม่บังคับ
<code>lat</code> / <code>lng</code>	ตัวเลข	พิกัด ไม่บังคับ
<code>map_url</code>	ข้อความ	ลิงก์ Google Maps ต้นฉบับ เก็บไว้อ้างอิง
<code>created_at</code> / <code>updated_at</code>	ข้อความ ISO	ระบบใส่ให้เอง
<code>deleted_at</code>	ข้อความ ISO	ว่าง = ยังไม่ถูกลบ / มีค่า = อยู่ในถังขยะ

แท็บ `Users` — 4 คอลัมน์: `username` , `password` , `role` (`admin` / `staff`), `is_active` (`TRUE` / `FALSE`)

แท็บ `Settings` — 2 คอลัมน์ เก็บแบบ key-value (คีย์-ค่า) มี 3 แถว: `system_name` , `municipality_lat` , `municipality_lng`

ทำไม Settings ถึงเก็บแบบ key-value แทนที่จะทำเป็นคอลัมน์แยก? เพราะเพิ่มค่าตั้งค่าใหม่ในอนาคตได้ โดยแค่เพิ่มแถว ไม่ต้องแก้โครงสร้างและไม่ต้องแก้โค้ดที่อ่านตาราง

การออกรหัสผู้ป่วยอัตโนมัติ: ระบบอ่านรหัสทั้งหมดที่มี หาเลขลำดับที่มากที่สุด แล้ว +1 เติมศูนย์ให้ครบ 4 หลัก ประกอบเป็น `P-{ปี ค.ศ.}-{เลข 4 หลัก}` และครอบด้วย `LockService` เพื่อกันกรณีมีคนกดเพิ่มพร้อมกันแล้วได้รหัสซ้ำ

6. การตัดสินใจออกแบบสำคัญ (เพื่ออาจารย์ถาม "ทำไมทำแบบนี้")

เรื่อง	ตัดสินใจว่า	เหตุผล
การลบข้อมูล	Soft Delete (ใส่ค่า <code>deleted_at</code> แทนการลบแถวจริง)	กันลบบลัด กู้คืนได้จากถังขยะ
จุดเริ่มนำทาง	ตรึงเป็นพิกัดเทศบาลเสมอ ไม่ใช่ตำแหน่ง GPS ผู้ใช้	ผู้เยี่ยมบ้านอาจไม่ได้ออกจากเทศบาลโดยตรง แต่ระบบต้องการเส้นทางอ้างอิงจากเทศบาลเป็นหลัก
สิทธิ์ใช้งาน	แบ่ง admin (แก้ไขได้) / staff (ดูอย่างเดียว)	staff ออกภาคสนามไม่ควรแก้ไขข้อมูลได้ ป้องกันข้อมูลผิดพลาด
การเก็บพิกัด	ให้วางลิงก์ Google Maps แล้วดึงพิกัดอัตโนมัติ แทนปักหมุดในแผนที่	ไม่ต้องขอ API Key ที่มีค่าใช้จ่าย เจ้าหน้าที่ก๊อปลิงก์จากมือถือได้เลย ง่ายกว่า
โหมดเดโม	ถ้ายังไม่ได้ต่อ Google Sheet ระบบใช้ข้อมูลจำลองแทน	ทดสอบ/สาธิตหน้าตาระบบได้ทันทีโดยไม่ต้อง setup Sheet ก่อน
ตัวระบุผู้ป่วย	ใช้ <code>patient_code</code> เป็นตัวระบุ ไม่เพิ่มคอลัมน์ <code>id</code> แยก	รหัสไม่ซ้ำกันอยู่แล้ว และทำให้ลิงก์อ่านรู้เรื่อง เช่น <code>patient-detail.html?id=P-2026-0001</code>
การค้นหา	ดึงข้อมูลทั้งหมดครั้งเดียว แล้วกรองด้วย JavaScript ฝั่งหน้าเว็บ	ข้อมูลชุดเล็ก (หลักสิบ-ร้อย) การกรองในเครื่องเร็วกว่ายิงถาม server ทุกตัวอักษร และประหยัด quota ของ Apps Script
หน่วงเวลาค้นหา	หน่วง 300 มิลลิวินาทีก่อนกรอง (debounce)	ถ้ากรองทุกครั้งที่เกิดแป้น หน้าจอจะกระตุก การรอให้ผู้ใช้หยุดพิมพ์ก่อนทำให้ลื่นกว่า
เก็บ Sheet ID	เก็บใน Script Properties ของ Apps Script ไม่เขียนลงในโค้ด	โค้ดอาจถูกอัปขึ้น GitHub รหัสชี้ตไม่ควรถัดไปด้วย
โลโก้	ฝังเป็นไฟล์ static ที่ <code>images/logo.png</code> ไม่เก็บใน Settings	SPEC กำหนดให้แท็บ Settings มีแค่ 3 ค่า และโลโก้แทบไม่มีการเปลี่ยน การทำระบบอัปโหลดไฟล์จะกินความจำเป็นมาก
ส่งข้อมูลแบบ POST	ส่งด้วย <code>Content-Type: text/plain</code> แทน <code>application/json</code>	Apps Script ไม่รองรับ CORS preflight (การที่เบราว์เซอร์ขออนุญาตข้ามโดเมนล่วงหน้า) ถ้าส่งเป็น JSON เบราร์เซอร์จะบล็อก จึงส่งเป็นข้อความธรรมดาแล้วให้ Apps Script แปลง JSON เอง

6.1 รายละเอียดทางเทคนิคที่ควรรู้ (เพื่ออาจารย์ถามลูก)

การดึงพิกัดจากลิงก์ Google Maps — ใช้ Regular Expression (รูปแบบการค้นหาคำข้อความ) 5 แบบ ไล่ตรวจทีละแบบ รองรับลิงก์แบบ @13.75,100.50 , !3d13.75!4d100.50 , ?q=13.75,100.50 และตัวเลขเปล่าๆ ที่ผู้ใช้คัดลอกมาวาง ถ้าไม่เข้าแบบไหนเลยจะขึ้นข้อความแนะนำวิธีแก้ ทำงานทั้งหมดในเบราว์เซอร์ไม่ต้องเรียก API ใดๆ

การป้องกัน XSS (การแอบฝังโค้ดอันตรายผ่านช่องกรอกข้อมูล) — ข้อความทุกตัวที่ดึงมาจาก Google Sheet ต้องผ่านฟังก์ชัน escapeHtml() ก่อนแสดงบนหน้าเว็บเสมอ เพื่อแปลงอักขระพิเศษ อย่าง < > ให้เป็นข้อความธรรมดา ไม่ให้เบราว์เซอร์ตีความเป็นคำสั่ง

การกันข้อมูลชนกัน — คำสั่งที่เขียนข้อมูล (create , update , delete , restore , updateSettings) ครอบด้วย LockService ซึ่งบังคับให้ทำทีละคำขอ ป้องกันกรณีเจ้าหน้าที่ 2 คนกดบันทึกพร้อมกัน แล้วข้อมูลทับกันหรือได้รหัสผู้ป่วยซ้ำ

การตรวจสอบข้อมูล 2 ชั้น — ตรวจที่หน้าเว็บก่อน (แสดงข้อความไทยใต้ช่องที่ผิดทันที ผู้ใช้ไม่ต้องรอ) และตรวจซ้ำที่ Apps Script อีกรอบ (validatePatient_) เพราะการตรวจฝั่งหน้าเว็บอย่างเดียว ผู้ใช้สามารถข้ามได้ ถ้าตั้งใจจะข้าม

ทุกหน้ามี 3 สถานะเสมอ — กำลังโหลด (แสดงกล่องเทาเรืองแสง) / ไม่มีข้อมูล (บอกว่าต้องทำอะไรต่อ) / ผิดพลาด (บอกสาเหตุเป็นภาษาไทย + ปุ่มลองใหม่) ไม่ปล่อยให้ผู้ใช้เห็นหน้าจอว่างเปล่าโดยไม่รู้ว่าจะเกิดอะไรขึ้น

Mobile-first — ออกแบบหน้าจอมือถือก่อน แล้วค่อยขยายเป็นจอใหญ่ ปุ่มทุกปุ่มสูงอย่างน้อย 44 พิกเซล ซึ่งเป็นขนาดมาตรฐานที่นิ้วกดได้แม่นยำ

6.2 การทดสอบที่ทำไปแล้ว

สิ่งที่ทดสอบ	ผล
ทุกฟังก์ชันบนคอมพิวเตอร์	✅ ผ่าน
การใช้งานบนมือถือจริง (ฟอร์ม / ปุ่มนำทาง / การจัดหน้าจอ)	✅ ผ่าน
สิทธิ์บัญชี staff — เมื่อกดจำกัด และพิมพ์ URL เข้าหน้า admin ตรงๆ แล้วถูกแจ้งเตือนกลับ	✅ ผ่าน
หน้าตั้งค่า — แก่ชื่อระบบและพิกัดเทศบาลแล้วบันทึกลงซีตจริง	✅ ผ่าน
การเพิ่ม/แก้ไข/ลบ/กู้คืนผู้ป่วย กับข้อมูลจริงในซีต	✅ ผ่าน
ลิงก์นำทางออกมาถูกต้อง จุดเริ่มเป็นพิกัดเทศบาล	✅ ผ่าน

7. ข้อจำกัดของระบบ (พูดตรงๆ ได้ถ้าอาจารย์ถาม จะดูน่าเชื่อถือกว่าอ้างว่าสมบูรณ์แบบ)

- รหัสผ่านเก็บใน Google Sheet แบบข้อความธรรมดา ไม่ได้เข้ารหัส — เหมาะกับงานสาธิต ไม่เหมาะกับการใช้จริงกับข้อมูลผู้ป่วยจริง
 - ไม่รองรับผู้ใช้พร้อมกันจำนวนมาก (Google Apps Script มีข้อจำกัดเรื่อง quota ต่อวัน)
 - ไม่มีการเข้ารหัสข้อมูลระหว่างส่ง-รับนอกเหนือจาก HTTPS มาตรฐาน
 - เหมาะกับข้อมูลจำนวนไม่เกินหลักพันแถว ถ้าจะขยายจริงต้องย้ายไปฐานข้อมูลระดับ production
 - ไม่มีการบันทึกว่าใครแก้ไขข้อมูลอะไรเมื่อไหร่ (audit log) รู้แค่ว่าแก้ล่าสุดเมื่อไหร่ แต่ไม่รู้ว่ามีใครแก้
 - ไม่มีระบบรีเซตรหัสผ่านด้วยตัวเอง ต้องให้ผู้ดูแลระบบเข้าไปแก้ในแท็บ `Users` ให้
 - การค้นหาข้อมูลทั้งหมดมาไว้ในเบราว์เซอร์ ถ้าข้อมูลเยอะมากจะโหลดช้าและกินหน่วยความจำ (เป็นผลจากการเลือกกรองฝั่งหน้าเว็บ ซึ่งแลกกับความเร็วในการค้นหาของข้อมูลขนาดเล็ก)
-

8. คำถามที่อาจารย์น่าจะถาม + แนวคำตอบ

Q: ทำไมไม่ใช้ฐานข้อมูลจริง เช่น MySQL? A: เลือกให้เหมาะกับขนาดงานและงบประมาณ ระบบยังคงหลักการ client-server เหมือนกัน แค่ใช้ Google Sheet แทน database server เพื่อไม่ต้องเสียค่าเช่า server และง่ายต่อการดูแลของเทศบาล

Q: ข้อมูลปลอดภัยแค่ไหน? A: มีระบบล็อกอินแยกสิทธิ์ admin/staff และการลบข้อมูลเป็นแบบ soft delete คุ้มครองได้แต่ยอมรับว่าเป็นระดับความปลอดภัยสำหรับงานสาธิต ถ้าจะใช้งานจริงต้องเพิ่มการเข้ารหัสข้อมูลและระบบยืนยันตัวตนที่แข็งแกร่งกว่านี้

Q: ทำไมไม่ใช้ Google Maps แบบปักหมุดในเว็บ? A: การปักหมุดต้องใช้ Google Maps JavaScript API ซึ่งต้องเปิดบัญชี Google Cloud และผูกบัตรเครดิต ระบบนี้ใช้วิธีวางลิงก์ + ดึงพิกัดจากลิงก์แทน ซึ่งฟรีและเจ้าหน้าที่ทำได้ง่ายกว่าจากมือถือ

Q: ระบบรองรับผู้ใช้กี่คนพร้อมกัน? A: Google Apps Script มีข้อจำกัดด้าน quota ของ Google เอง เหมาะกับทีมงานขนาดเล็ก (เจ้าหน้าที่เทศบาลไม่กี่คน) ไม่ได้ออกแบบมาสำหรับผู้ใช้จำนวนมากพร้อมกัน

Q: เขียนโค้ดเองทั้งหมดไหม? A: ออกแบบโครงสร้างระบบ, ตัดสินใจเลือกเทคโนโลยี, กำหนด flow การทำงาน และตรวจสอบ/ทดสอบทุกจุดด้วยตัวเอง ใช้เครื่องมือช่วยเขียนโค้ดบางส่วนเพื่อความเร็ว เหมือนการใช้ตัวช่วยเขียนโค้ด (เช่น autocomplete) ทั่วไปในการพัฒนาซอฟต์แวร์ปัจจุบัน — สามารถอธิบายการทำงานทุกส่วนได้เพราะเป็นคนกำหนดโจทย์และตรวจสอบผลลัพธ์เองทั้งหมด